

Agent > Task Groups

Task Groups

When Agent is in planning mode, large and complex tasks are handled with Task Groups, which break down these problems into smaller, more approachable units of work. Oftentimes, Agent will work on multiple parts of the greater task at the same time, and task sections are how these changes are presented to the user. Here is an example task group.


Proceeded with

Implementation Plan

Auto-proceeded by the agent under your review policy.

Agent Decides

> Thought for <1s

Great! The plan has been approved. I will now proceed with the implementation. First, I'll create the `utils` directory and the `color.ts` file within it.

Implementing Card Color Randomization

The implementation plan was approved. The `ts color.ts` utility file has been created.

Files Edited


Card.tsx

TS color.ts

Progress Updates

Expand all

1 Creating the `utils` directory and `color.ts` file

2 Modifying the `Card.tsx` component

The top component of the task group specifies the overarching goal of this task as well as summarizes the changes made within this unit of work. There is also a section of edited files for quick user audit of changes: click on the file pill and you will view the current state of the changed files.


Proceeded with

Implementation Plan

Auto-proceeded by the agent under your review policy.

Agent Decides

> Thought for <1s

Great! The plan has been approved. I will now proceed with the implementation. First, I'll create the `utils` directory and the `color.ts` file within it.

Implementing Card Color Randomization

The implementation plan was approved. The `color.ts` utility file has been created.

Files Edited


Card.tsx

TS color.ts

Progress Updates

Collapse all

1 Creating the `utils` directory and `color.ts` file

```

~/src $ mkdir utils
mkdir: utils: File exists

```

Ran background terminal command
Open Terminal
Exit code 1
Auto

Created `color.ts`
Open diff

> Thought for 1s

Great, the color utility file has been created. Now, I'll modify the `Card.tsx` component to use this new function and apply the random colors to the cards.

2 Modifying the `Card.tsx` component

Edited `Card.tsx`
Open diff

The changes have been applied. Now, I will run the application to verify that the card colors are randomized and the text is legible.


Card.tsx

```

1  import type { Card as CardType } from '../gameLogic';
2  import { getRandomColor } from '../utils/color';
3
4  interface CardProps {
5    card: CardType;
6    isNew?: boolean;
7    index?: number;
8  }
9
10 export default function Card({ card, isNew, index = 0 }: CardProps) {
11   if (card.hidden) {
12     return <div className="card card-back ${isNew ? 'card-dealing' : ''}" style={{ animationDelay: `${index * 0.15}s` }}></div>;
13   }
14   const randomColor = getRandomColor();
15
16   return (
17     <div
18       className="card ${isNew ? 'card-dealing' : ''}"
19       style={{ animationDelay: `${index * 0.15}s`, backgroundColor: randomColor, color: 'black', textShadow: '0 0 2px white' }}
20     >
21       <div style={{ fontSize: '1.2em', marginBottom: '-8px' }}>{card.rank}</div>
22       <div style={{ fontSize: '2em' }}>{card.suit}</div>
23     </div>
24   );
25 }
26
27

```

Agent > Task Groups

✓ Proceeded with Implementation Plan

Auto-proceeded by the agent under your review policy. Agent Decides ^

> Thought for <1s

Great! The plan has been approved. I will now proceed with the implementation.
First, I'll create the `utils` directory and the `color.ts` file within it.

Implementing Card Color Randomization
The implementation plan was approved. The `ts color.ts` utility file has been created.

Files Edited
 Card.tsx `ts color.ts`

Progress Updates Collapse all v

1 Creating the `utils` directory and `color.ts` file

```
.../src $ mkdir utils  
mkdir: utils: File exists
```

Ran background terminal command [Open Terminal](#) · Exit code 1 Auto ^

Created `ts color.ts` +5 -0 Open diff

> Thought for 1s

Great, the color utility file has been created. Now, I'll modify the Card.tsx component to use this new function and apply the random colors to the cards.

2 Modifying the `Card.tsx` component

Edited Card.tsx +6 -3 Open diff

The changes have been applied. Now, I will run the application to verify that the card colors are randomized and the text is legible.

Sometimes, there are pending steps, such as browser setup or terminal commands requiring approval, that are created inside these progress updates. In this case, instead of expanding all of the updates, Agent provides a special section at the end of the task group where you can review these pending steps accordingly.

<https://antigravity.google/docs/task-groups>

2/3

 Agent > Task Groups

Initiated verification of the card color randomization feature.

Files Edited

 Task

Progress Updates

Expand all <

1

Checking file integrity of modified components

 1 Step Requires Input

Collapse v

workspace \$ ls -l src/components/Card.tsx src/utis/color.ts

Run command?

Reject

Accept

< Skills

Browser Subagent >